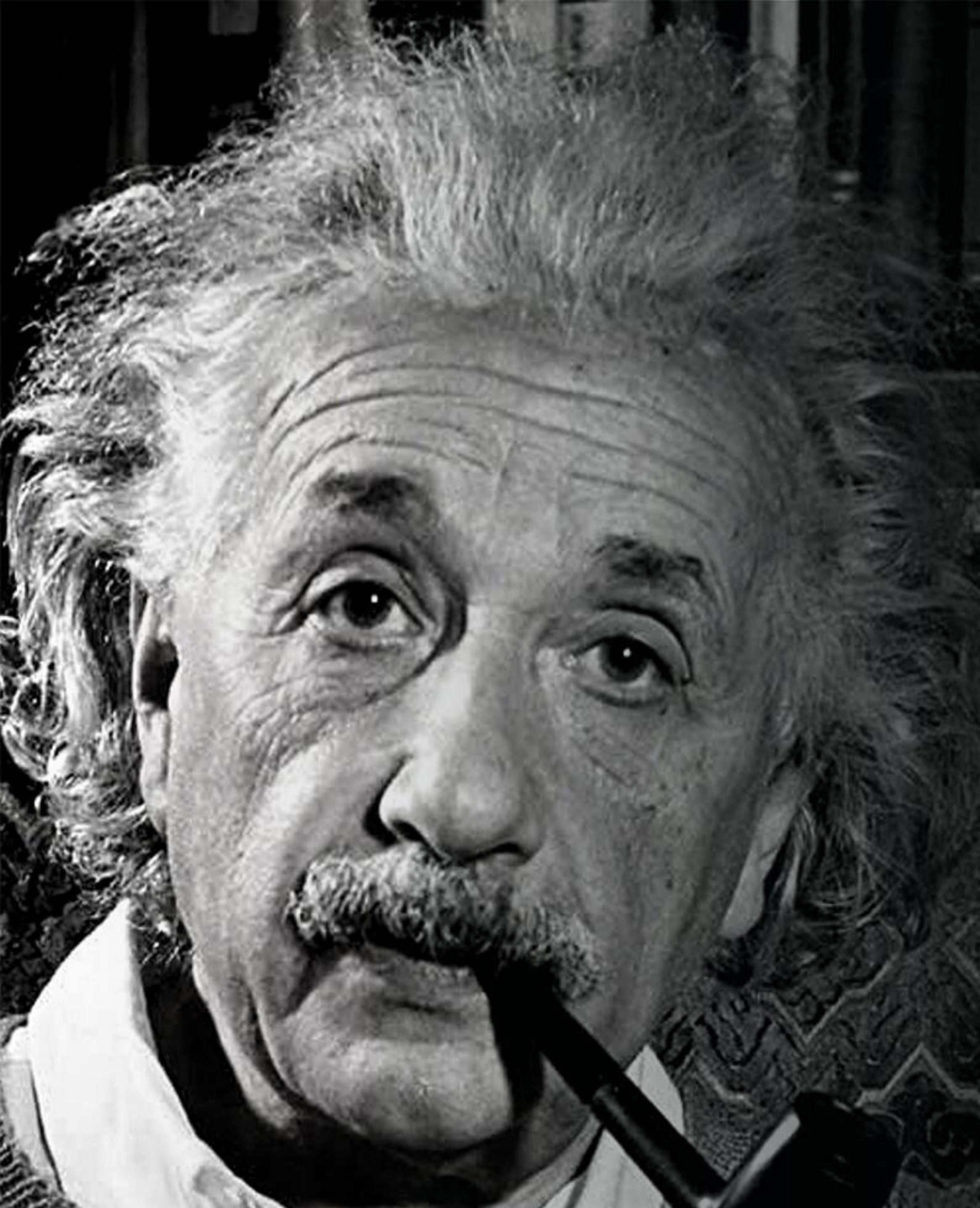

The more I learn, the more I realize how much I don’t know.

ALBERT EINSTEIN

Zain AI Bootcamp

3-Day Learning Flow, LLM Foundations, Project Roadmap & Setup

Python • SQL • LangChain • RAG • Agents • MCP • Capstone

Cognit DX

Software Engineer
2005 - 2010

absolutdata
-an lirfogain company

Data Scientist
2012 - 2012

2010

DELL Technologies

Data Scientist
2012 - 2014

accenture

Lead Data Scientist
2014 - 2021

MAJID AL FUTTAIM

Data Science Manager
2021 - 2022

DECODING DATA SCIENCE

Founder
2022 - Present

EXPERIENCE BUILDING AND LECTURING AT

accenture DELL Technologies MAJID AL FUTTAIM Microsoft

HARVARD BUSINESS REVIEW Yale GAP

Why this workshop matters

From simple prompting to enterprise AI applications

The shift

Most people can ask ChatGPT questions. Business AI needs systems that can access data, use tools, retrieve evidence, reason through steps, and recommend actions.

The telecom reality

Customer care, retention, billing, network operations, and campaign teams need AI grounded in Customer 360 data — not generic answers.

The outcome

Participants will build from database basics to tools, SQL agents, RAG, multi-agent workflows, MCP, and a final capstone prototype.

Workshop promise: learn what to use, when to use it, and how to build it step by step.

Summary: GenAI Workshop Steps

1. Define the Use Case: Choose domain (HR/Support), define goals
2. Select the Right LLM: Match model to task complexity and cost
3. Configure the GenAI Model: Adjust temperature, tokens, system prompt
4. Prompt Engineering: Use role/task formatting and few-shot examples
5. Vector DB: Embed documents, enable semantic search
6. RAG Pipeline: Combine search with prompts for grounded answers
7. Build Interface: Use FastAPI, Gradio, or Streamlit for UI
8. Deploy: Use Docker, APIs, GitHub, and environment variables

Step 0: Create your OpenAI API key

Do this before running the AI notebooks

1. Go to platform.openai.com and sign in.
2. Open or create the correct project for this workshop.
3. Open Project Settings → API Keys.
4. Click “+ Create new secret key”.
5. Copy the secret key immediately and store it securely.
6. Do not share the key in chat, slides, GitHub, or screenshots.

Security rule

OpenAI shows the full secret key only when it is created. If participants lose it, they should create a new key and update their notebook or app configuration.

Step 0: Add the API key safely

The key should be stored as a secret or environment variable

Google Colab

Use Colab Secrets.

Secret name:
OPENAI_API_KEY

Notebook code reads the key from the secure environment instead of hard-coding it.

Hugging Face Space

Go to Space Settings → Secrets.

Add secret name:
OPENAI_API_KEY

Restart the Space after changing secrets if needed.

Local Python

Use an environment variable.

Example variable name:
OPENAI_API_KEY

Never commit the value into GitHub or a notebook.

Code pattern: os.environ["OPENAI_API_KEY"]

FOUNDATION

Before building agents, understand the LLM

Participants first need a simple mental model for model choice, reasoning, generation, and parameter tuning.

LLM landscape: don’t look for one “best” model

Choose models based on task, data, cost, latency, and reliability needs

OpenAI GPT

general AI apps
reasoning + tools

Claude

writing, coding,
long-form reasoning

Gemini

multimodal +
Google ecosystem

Llama

open-weight
deployment options

Mistral

fast + efficient
open models

Qwen / DeepSeek

strong open
model ecosystem

Decision lens: quality • reasoning • context length • speed • cost • privacy • deployment constraints

Reasoning models vs generation models

Different model behaviors support different enterprise tasks

Model behavior	Best for	Example tasks
Generation-focused	Fast fluent output	emails, summaries, drafts, simple Q&A
Reasoning-focused	Multi-step thinking	planning, coding, analysis, troubleshooting
Tool-using	Action workflows	SQL tools, APIs, agents, MCP tools
Retrieval-grounded	Company knowledge	RAG, policy assistant, support assistant

Teaching message: business AI often combines generation + reasoning + retrieval + tools.

Cognit DX

Model	Category	Ideal For	Strengths	Input / Output Cost
GPT-5.5	Frontier / Flagship	Complex reasoning, coding, agents, professional work	Strongest general model, long-context work, advanced tool use	**$5.00 / $30.00**
GPT-5.5 Pro	Frontier / Pro	Deep reasoning, high-precision analysis, critical tasks	More precise and capable version of GPT-5.5	**$30.00 / $180.00**
GPT-5.4	Frontier	Coding, business workflows, advanced AI apps	More affordable frontier model for professional work	**$2.50 / $15.00**
GPT-5.4 Pro	Frontier / Pro	High-accuracy reasoning and professional workloads	Smarter, more precise GPT-5.4 variant	Pricing varies / check API pricing
GPT-5.4 Mini	Frontier Mini	High-throughput apps, coding assistants, sub-agents	Strong mini model, lower latency, lower cost	**$0.75 / $4.50**
GPT-5.4 Nano	Frontier Nano	Classification, extraction, routing, ranking, simple agents	Cheapest GPT-5.4-class model	**$0.20 / $1.2**

LLM generation controls

Small parameter changes can affect creativity, reliability, length, and repetition

Temperature	Randomness / creativity
Top-p	Diversity of likely tokens
Top-k	Limits token choices
Max tokens	Controls output length
Repetition penalty	Reduces repeated wording

Workshop rule

For enterprise use cases, start controlled.

Use lower creativity for factual answers, RAG, SQL outputs, coding, and customer-care messages. Increase creativity only when the task needs ideation.

Live demo: LLM Generation Controls Space

Use the Space to compare model behavior dynamically

Demo prompt

Write a short customer-care message for a telecom customer who is unhappy about billing confusion.

Run it multiple times with different settings and compare output stability, tone, and detail.

1. Open the Space link
2. Enter the business prompt
3. Run with low temperature
4. Run with higher temperature
5. Compare the outputs
6. Discuss where each setting is useful

Space link: huggingface.co/spaces/decodingdatascience/LLMGenerationControls

WORKSHOP ROADMAP

The 3-day journey and 9-class structure

Every class adds one capability to the final enterprise AI application architecture.

3-day learning journey

Each day moves participants one level closer to a working enterprise AI prototype

Day 1
Data → Tools → SQL

Class 1: Database Foundation
Class 2: LangChain Tools Agent
Class 3: Natural Language SQL Agent

Output: participants can query data, create tools, and ask analytical questions.

Day 2
RAG → Multi-Agent → MCP

Class 4: RAG Assistant
Class 5: Multi-Agent Copilot
Class 6: MCP Enterprise Integration

Output: participants understand retrieval, specialist agents, and reusable tool access.

Day 3
Plan → Build → Present

Class 7: Capstone Planning
Class 8: Build Sprint
Class 9: Final Evaluation

Output: teams build and present a database-backed AI prototype.

Full 9-class roadmap

The workshop progresses from data understanding to final solution review

1 Database → 2 Tools → 3 SQL Agent

4 RAG → 5 Multi-Agent → 6 MCP

7 Planning → 8 Build Sprint → 9 Evaluation

Storyline: understand data → expose capabilities → reason and retrieve → coordinate workflows → build and present.

Day 1 technical path

Build the foundation: database, Python functions, tools, and SQL analytics

Customer 360 DB

Understand tables, rows, and business meaning

→

Python Functions

Reusable customer profile, plan, churn, billing logic

→

LangChain Tools

Wrap functions as tools the agent can call

→

Customer Care Agent

Answer customer-specific questions using tools

→

SQL Agent

Ask broader analytical questions in natural language

Day 1 output: participants can connect the database, query it, create tools, and run their first AI-powered data workflows.

Day 2 technical path

Move from single agents into enterprise-style AI architecture

Rows → Documents

Convert plans, campaigns, complaints, support notes into text

RAG Assistant

Retrieve relevant context before answering

Specialist Agents

Split customer, SQL, RAG, and message responsibilities

Supervisor Agent

Coordinate specialist agents and synthesize final report

MCP Tools

Expose selected capabilities through an enterprise-style server

Day 2 output: participants understand how SQL, RAG, agents, and MCP can work together in a realistic enterprise workflow.

Day 3 project path

Teams turn the learning into a focused AI prototype

Choose Problem

One clear telecom business problem

→

Select Architecture

Tools, SQL Agent, RAG, Multi-Agent, or MCP

→

Build MVP

Smallest useful working demo first

→

Test Main Prompt

Validate database-backed output

→

Present Story

Problem, data, architecture, demo, value

Day 3 rule: do not try to build everything. Build the smallest useful demo, then improve it.

The 9 steps of the final project

Participants will follow a clear project creation process

1 **Understand database**
Know available data

2 **Write SQL queries**
Extract business insights

3 **Create Python functions**
Build reusable logic

4 **Convert to tools**
Make agent-ready capabilities

5 **Build LangChain agent**
Customer-care assistant

6 **Add SQL Agent**
Business analytics assistant

7 **Add RAG**
Grounded recommendations

8 **Add Multi-Agent / MCP**
Enterprise-style workflow

9 **Build capstone demo**
Final AI prototype

Architecture decision guide

Choose the right pattern based on the project need

Project need	Best pattern
Retrieve one customer’s information	**LangChain Tools**
Ask broad analytical questions	**SQL Agent**
Recommend plans, offers, or campaigns	**RAG**
Combine multiple specialist tasks	**Multi-Agent**
Expose reusable enterprise tools	**MCP**
Build final prototype	**Capstone workflow**

Simple rule: the architecture should match the question, not the other way around.

What makes a good capstone project?

A strong demo is practical, grounded, and easy to explain

Clear

solves one specific business problem

Demoable

can be shown in 3–5 minutes

Data-grounded

uses Customer 360 database evidence

Useful

helps a real business user

Realistic

can be built in available time

Explainable

architecture can be described clearly

Trainer cue: ask teams to cut scope until the demo becomes obvious.

Final demo expectations

Each team will present a clear 5-minute solution story

1. Project title

2. Business problem

3. Target user

4. Data used

5. AI architecture

6. Live demo

7. Business value

8. Future improvements

Final message

The goal is not to show the most complex architecture. The goal is to show a useful, explainable, database-backed AI prototype that solves a real telecom business problem.

Workshop resources: GitHub + live labs

Participants will use one shared resource hub throughout the 3 days

GitHub Repository

Add repository link here:

https://github.com/arshad831/zain_2026

Includes notebooks, database file, setup instructions, code templates, capstone templates, and troubleshooting notes.

Hugging Face Space

LLM Generation Controls:

https://huggingface.co/spaces/decodingdatascience/LLMGenerationControls

Use it to compare generation behavior across temperature, top-p, top-k, max tokens, and repetition controls.

Trainer cue: remind participants to bookmark both links before Class 1 begins.

Mohammad Arshad Ahmad
United Arab Emirates
Emirates Dubai
Cognit
110001
Number: 4098 8728 55688948

Expiry date: 05/28

CVV:

https://sqlitebrowser.org
/dl/

Submit your Project here

Reference links for setup and learning

Use these links during setup and self-practice

Workshop GitHub repository

OpenAI API key page

https://platform.openai.com/api-keys

OpenAI API key safety

Use environment variables and never commit keys to GitHub.

LLM Generation Controls Space

https://huggingface.co/spaces/decodingdatascience/LLMGenerationControls

Trainer note: replace the GitHub placeholder before presenting.

Download VS Code

https://code.visualstudio.com/download

Submit Project

https://forms.gle/CCUU
mCYYA